Unit II

Disjoint Sets Introduction

1. A tree is used to represent each set and the root to name a set.
2. Each node points upwards to its parent
3. Two sets A and B are said to be disjoint if A — B — f. i.e., there is no common element in both A and B
4. A collection of disjoint sets is called a disjoint set forest
5. An array can be used to store parent of each element

Let £ - (S1 , S2..... Sk) be a collection of dynamic disjoint-sets of the elements and let x and y be any two elements. We want to support:

Make-Set(x): create a set containing x
Find(x): return which set x belongs
Union(x. y): merge the sets containing x and containing y into one

Example:
n-11. 4 sets
s1 = (1,7,10,11) . s2 - (2.3.5.6), s3 = (4,8) and s4 = (9)

Disjoint set operations

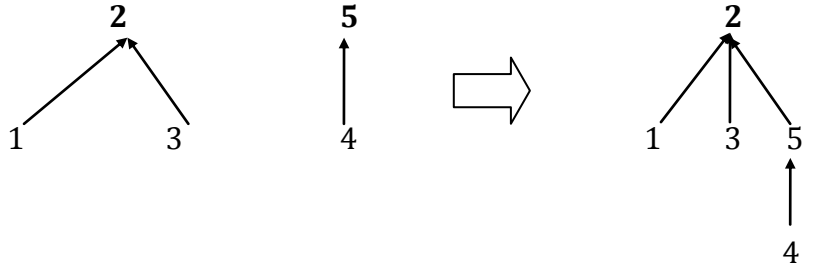
Two important operations performed on disjoint sets are:

1. Union
2. Find

Disjoint set operations Union

1. If S1 and S2 are two disjoint sets, their union S1 U S2 is a set of all elements x such that x is in either S1 or S2
2. As the sets should be disjoint S1 U S2 replaces S1 and S2 which no longer exist
3. Union is achieved by simply making one of the trees as a sub tree of other i.e to set parent field of one of the roots of the trees to other root

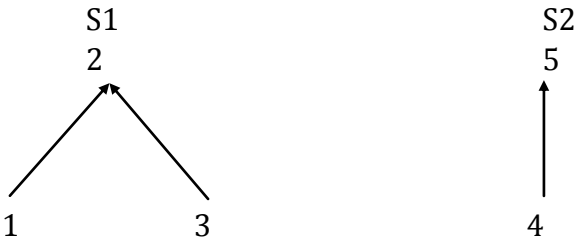
<u>Example:</u> **s1** **s2** **s1 union s2**



2. Find:

Given an element x, to find the set containing it

Example:



find(3) means in which set the element 3 is there i.e., 3 is in S1
find(5) means in which set the element 5 is there i.e., 5 is in S2

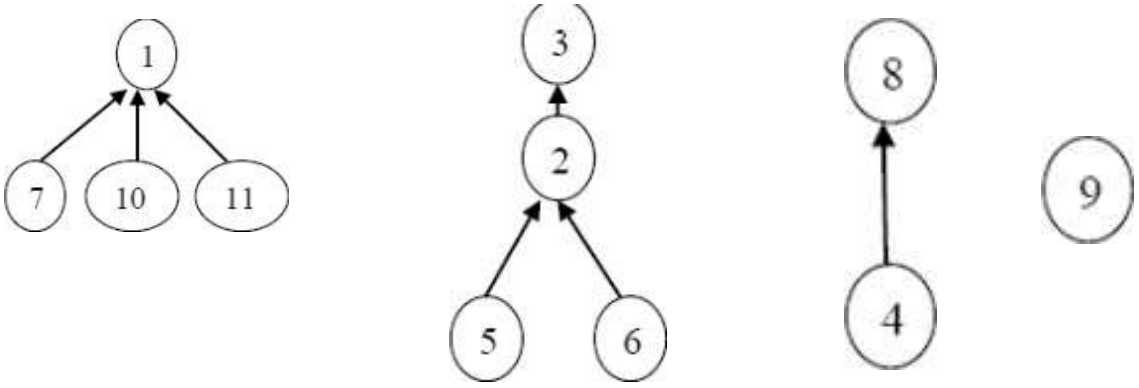
Disjoint Sets:

Set Representation:

1. An array can be used to store parent of each element
2. The ith element of this array represents the tree node that contains the element I and it gives the parent of that element
3. Root node has a parent -1
4. An array P|1:n] can be taken for all n elements in the forest
5. Element at the root node is taken as the name of the set

Example: n-11. 4 sets S1 = (1,7, 10, 11) S2 = (2, 3, 5, 6) S3 = (4, 8) and S4 - {9)

UNION FIND ALGORITHMS



i	1	2	3	4	5	6	7	8	9	1	11
P	-1	3	-1	8	2	2	1	-1	-1	1	1

UNION FIND ALGORITHMS

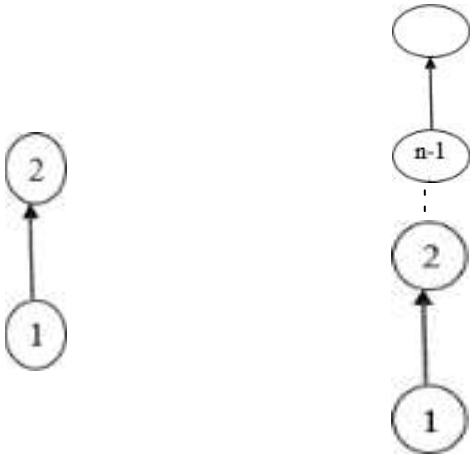
Union:

```
Algorithm  simpleUnion (i,j)
{
                P[ i] : = j;        }
```

Find:

```
Algorithm Simple find (i)
{
        while(p[i])>=0) do
        {
                i := P[ i];
        }
        return I;
}
```

In a worst-case scenario SimpleUnion() and SimpleFind() perform badly. Suppose we start with the single element sets{1},{2},...,{n} , then execute the following sequence of unions and finds UNION(1,2), UNION(2,3),...,UNION(n —1,n).FIND(1),FIND(2),...,FIND(n).



This sequence results in a skewed tree
Time complexity for 1 union = 1
Time complexity for n-1 union = n-1
Therefore Tall unions = 0(n)
Time complexity for 1 find = i
Time complexity for n find = 1+2+....+ n-1 + n = n(n+1)/2
Therefore Tall finds = 0 (n2)

Disjoint set operations weighted Union

1. Simple Union leads to high time complexity in some cases
2. Weighted union is a modified union algorithm with weighted rules
3. Widely used to analyze the time complexity of an algorithm in average case
4. Weighted union deals with making the smaller tree a sub tree of the larger

5. If the number of nodes in the tree with root I is less than the no of nodes with root j,then make j the parent of I,otherwise make I the parent of j
6. Count of nodes can be places as a negative number in the P[i]value of the root i.

```
Weighted Union(I,j)
{
        Temp := P[ i ]+P  [ j ];
        If (P[ i ]>P  [ j ]) then
        {
                P[ i ] = j;
                P [ j ] = temp;
        }
        Else
        {
                P[ j ] = i;
                P [ i ] = temp;
        }
}
```

Collapsing Find

1. SimpleFind() leads to high time complexity in some cases
2. CollapsingFind() is a modified version of simpleFind()
3. If j is a node on the path from I to its parent and p[i] != root ,then set j to the root

```
Algorithm Collapsing Find(i)
{
        r:=i
        while (P [r] >0 )do {
                r:= P[i];
        }
        while ( i !=r )do {
                j:= P[i];
                P [ i ]:= r;
                i  := j;
        }
        Return r;
}
```

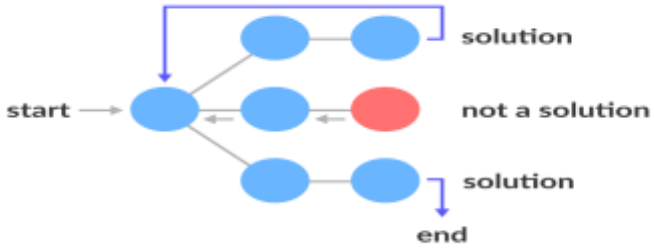
BACKTRACKING

Backtracking is a technique based on algorithm to solve problem. It uses recursive calling to find the solution by building a solution step by step increasing values with time. It removes the solutions that doesn't give rise to the solution of the problem based on the constraints given to solve the problem.

Backtracking algorithm is applied to some specific types of problems,Decision problem used to find a feasible solution of the problem. Optimization problem used to find the best solution that can be applied. Enumeration problem used to find the set of all feasible solutions of the problem. In backtracking problem, the algorithm tries to find a sequence path to the solution which has some small checkpoints from where the problem can backtrack if no feasible solution is found for the problem.

State Space Tree

A space state tree is a tree representing all the possible states (solution or nonsolution) of the problem from the root as an initial state to the leaf as a terminal state.

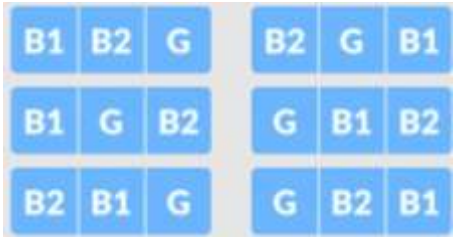


Example Backtracking Approach

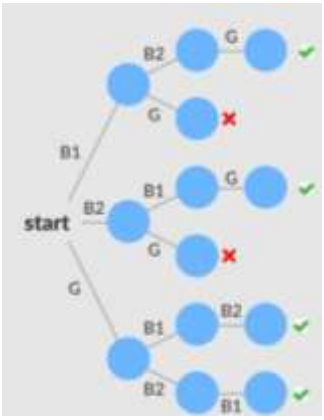
Problem: You want to find all the possible ways of arranging 2 boys and 1 girl on 3 benches. Constraint: Girl should not be on the middle bench.

Solution: There are a total of 3! = 6 possibilities. We will try all the possibilities and get the possible solutions. We recursively try all the possibilities.

All the possibilities are:



The following state space tree shows the possible solutions.



Backtracking Algorithm Applications

- To find all Hamiltonian Paths present in a graph.
- To solve the N Queen problem.
- Maze solving problem.
- The Knight's tour problem.

Backtracking Algorithm

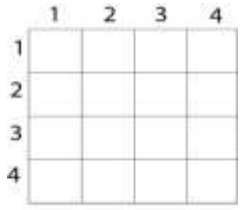
```
Backtrack(x)
        if x is not a solution
                Return false
                if x is a new solution
                        add to list of solutions
        backtrack (expand x)
```

N Queen Problem

The N Queen is the problem of placing N chess queens on an N×N chessboard so that no two queens attack each other.

It can be seen that for n =1, the problem has a trivial solution, and no solution exists for n =2 and n =3. So first we will consider the 4 queens problem and then generate it to n - queens problem.

Given a 4 x 4 chessboard and number the rows and column of the chessboard 1 through 4.



4x4 chessboard

Since, we have to place 4 queens such as q1 q2 q3 and q4 on the chessboard, such that no two queens attack each other. In such a conditional each queen must be placed on a different row, i.e., we put queen "i" on row "i."

Now, we place queen q1 in the very first acceptable position (1, 1). Next, we put queen q2 so that both these queens do not attack each other. We find that if we place q2 in column 1 and 2, then the dead end is encountered. Thus the first acceptable position for q2 in column 3, i.e. (2, 3) but then no position is left for placing queen 'q3' safely. So we backtrack one step and place the queen 'q2' in (2, 4), the next best possible solution. Then we obtain the position for placing 'q3' which is (3, 2). But later this position also leads to a dead end, and no place is found where 'q4' can be placed safely. Then we have to backtrack till 'q1' and place it to (1, 2) and then all other queens are placed safely by moving q2 to (2, 4), q3 to (3, 1) and q4 to (4, 3). That is, we get the solution (2, 4, 1, 3). This is one possible solution for the 4-queens problem. For another possible solution, the whole method is repeated for all partial solutions. The other solutions for 4 - queens problems is (3, 1, 4, 2) i.e

	1	2	3	4
1			q_1	
2	q_2			
3				q_3
4		q_4		

The implicit tree for 4 - queen problem for a solution (2, 4, 1, 3) is as follows:

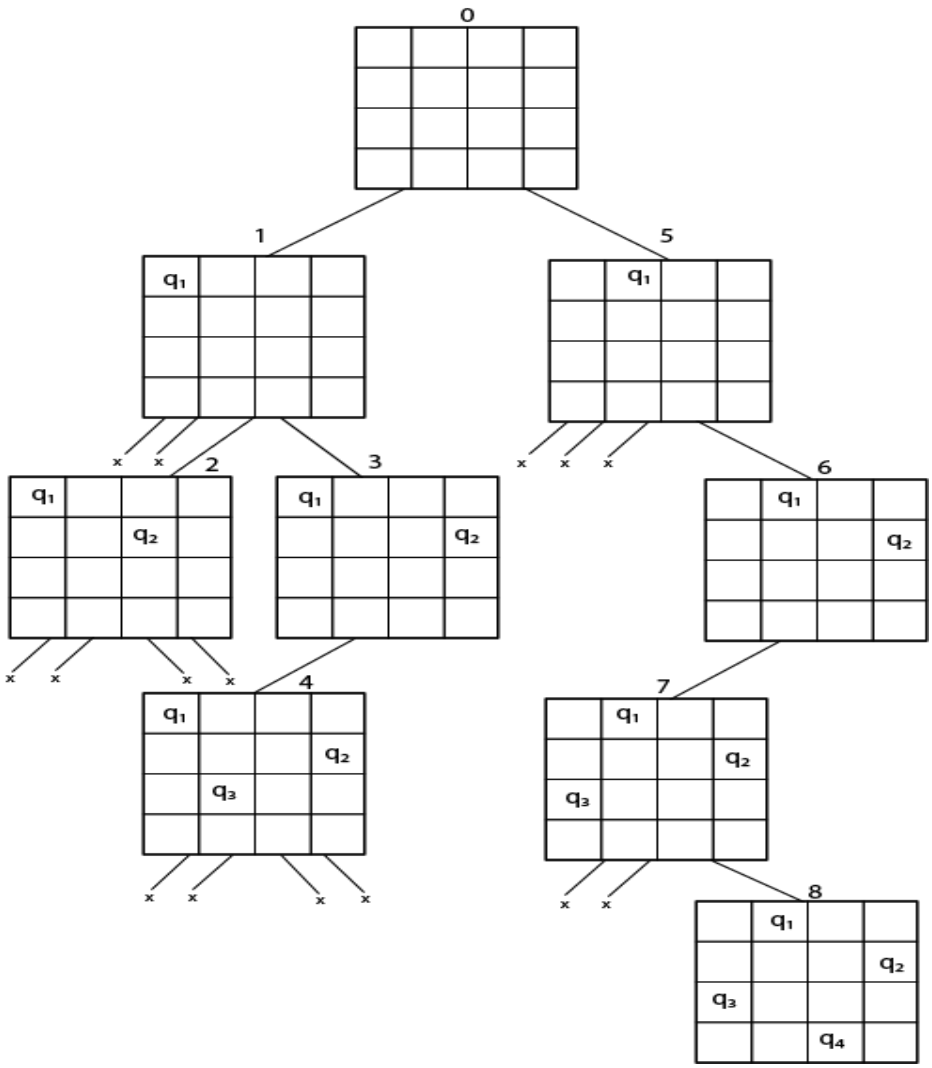
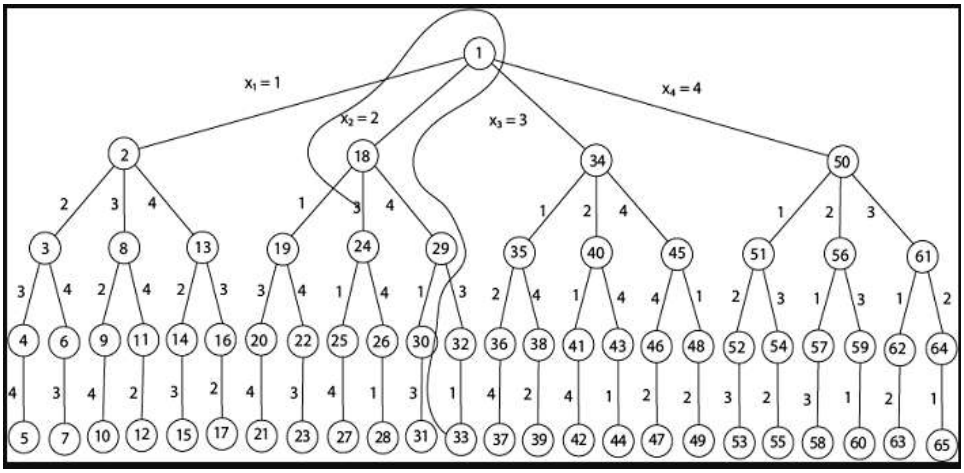


Fig shows the complete state space for 4 - queens problem. But we can use backtracking method to generate the necessary node and stop if the next node violates the rule, i.e., if two queens are attacking.



4 - Queens solution space with nodes numbered in DFS

It can be seen that all the solutions to the 4 queens problem can be represented as 4 - tuples (x_1, x_2, x_3, x_4) where x_i represents the column on which queen "q_i" is placed.

One possible solution for 8 queens problem is shown in fig:

	1	2	3	4	5	6	7	8
1				Q1				
2						Q2		
3								Q3
4		Q4						
5							Q5	
6	Q6							
7			Q7					
8					Q8			

1. Thus, the solution **for** 8 -queen problem **for** (4, 6, 8, 2, 7, 1, 3, 5).
2. If two queens are placed at position (i, j) and (k, l).
3. Then they are on same diagonal only **if** (i - j) = k - l or i + j = k + l.

The first equation implies that j - l = i - k.
The second equation implies that j - l = k - i.
Therefore, two queens lie on the duplicate diagonal if and only if |j-l|=|i-k|

Place (k, i) returns a Boolean value that is true if the kth queen can be placed in column i. It tests both whether i is distinct from all previous costs $x_1, x_2, \ldots x_{k-1}$ and whether there is no other queen on the same diagonal.

Using place, we give a precise solution to then n- queens problem.

```
Place (k, i)
  {
   For j ← 1 to k - 1 do
        if (x [j] = i)  or (Abs x [j]) - i) = (Abs (j - k))  then
                return false;
   return true;
}
```

Place (k, i) return true if a queen can be placed in the kth row and ith column otherwise return is false.
x [ ] is a global array whose final k - 1 values have been set. Abs (r) returns the absolute value of r.

```
N - Queens (k, n)
{
        For i ← 1 to n do
                if Place (k, i) then
                {
                        x [k] ← i;
                        if (k ==n) then
                                write (x [1....n));
                        else
                N - Queens (k + 1, n);
                }
}
```

TIME COMPLEXITY OF N-QUEEN'S PROBLEM IS

	CONSTANT	LOG	LINEAR	N-LOG-N	QUADRATIC	CUBIC	EXPONENTIAL
N	O(1)	O(LOG N)	O(n)	O(n log n)	$O(n^2)$	$O(n^3)$	$O(2^n)$
1	1	1	1	1	1	1	2
2	1	1	2	2	4	8	4
4	1	2	4	8	16	64	16
8	1	3	8	24	64	512	256

Subset Sum Problem

Subset sum problem is the problem of finding a subset such that the sum of elements equals a given number. The backtracking approach generates all permutations in the worst case but in general, performs better than the recursive approach towards subset sum problem.

A subset A of n positive integers and a value **sum** is given; find whether or not there exists any subset of the given set, the sum of whose elements is equal to the given value of sum.
Example:

- Given the following set of positive numbers: { 2, 9, 10, 1, 99, 3}
- We need to find if there is a subset for a given sum say 4: { 1, 3 }
- For another value say 5, there is another subset:{ 2, 3}
- Similarly, for 6, we have {2, 1, 3} as the subset.
- For 7, there is no subset where the sum of elements equal to 7.

This problem can be solved using following algorithms:

- Recursive method
- Backtracking
- Dynamic Programming

In Backtracking algorithm as we go down along depth of tree we add elements so far, and if the added sum is satisfying explicit constraints, we will continue to generate child nodes further. Whenever the constraints are not met, we stop further generation of sub-trees of that node, and backtrack to previous node to explore the nodes not yet explored. We need to explore the nodes along the breadth and depth of the tree. Generating nodes along breadth is controlled by loop and nodes along the depth are generated using recursion (post order traversal).

Steps:

- Start with an empty set
- Add the next element from the list to the set
- If the subset is having sum/weight M/W, then stop with that subset as solution.
- If the subset is not feasible or if we have reached the end of the set, then backtrack through the subset until we find the most suitable value.
- If the subset is feasible (sum/weight of subset < M or W) then go to step 2.
- If we have visited all the elements without finding a suitable subset and if no backtracking is possible then stop without solution.

Algorithm sumofSubsets(s.k,r)

```
{       X[k]=1
        If(s+w[k]=m) then
                Write(x[1:k])
        Else if(s+w[k] +w[k+1] <=m )then
                Sumofsubsets(s+w[k],k+1,r-w[k]);
        If((s+r-w[k]>= m ) and (s+w[k+1]<=m))then
                X[k]=0;
                Sumofsubsets(s,k+1,r-w[k]);
}
```

Example

1)Consider the following array/ list of integers: {1, 3, 2}
We want to find if there is a subset with sum 3.
Note that there are two such subsets {1, 2} and {3}.
We will follow our backtracking approach.

Consider our empty set {}
We add 1 to it {1} (sum = 1, 1 < 3)
We add 3 to it {1, 3} (sum = 4, 4 > 3, not found)
We remove 3 from it {1} (sum = 1, 1 < 3)
We add 2 to it {1, 2} (sum = 3, 3 == 3, found)
We remove 2 and see that all elements have been considered.
Following diagram captures the idea:

{}->{1}->{1,3}(found) ->{1}->{1,2}(found)->{1}(end)

The sum of s' is too large i.e. $s' + S_i + 1 > X$
The sum of s' is too small i.e. $s' + \sum_{j=i+1}^{n} Sj < X$

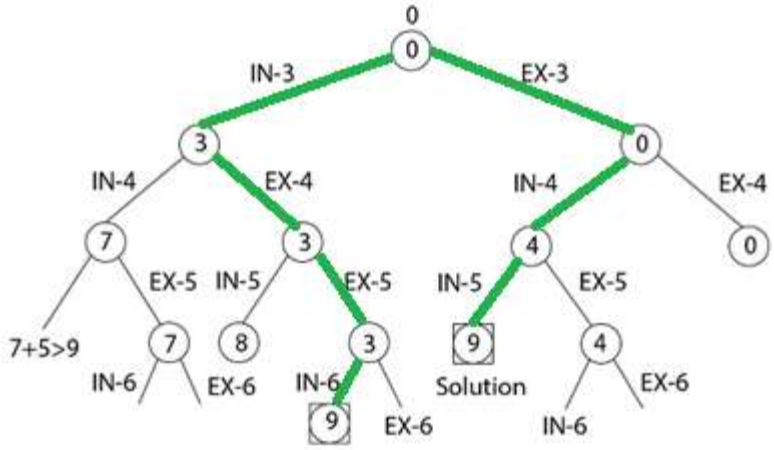
2)Example: Given a set S = (3, 4, 5, 6) and X =9. Obtain the subset sum using Backtracking approach.

Solution:

Initially S = (3, 4, 5, 6) and X =9.

S'= (Ø)

The implicit binary tree for the subset sum problem is shown as fig:

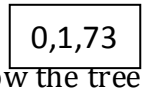


Example 3:Total values N=6,

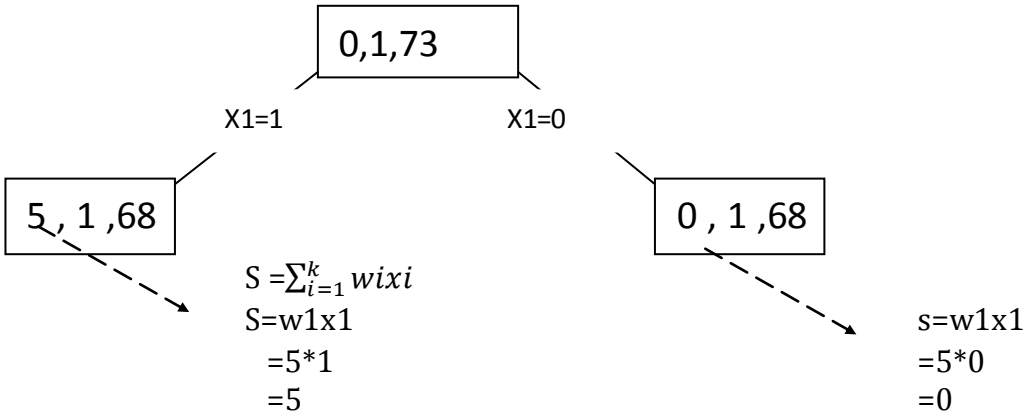
Total sum m=30,
Set of values W[1:6]=(5,10,12,13,15,18)
Solution : now let us consider x1=5, x2=10, x3=12, x4=13, x5=15, x6=18
As we know that

R=$\sum_{i=k+1}^{n} wi$ => 5+10+12+13+15+18=73
S =$\sum_{i=1}^{k} wixi$

By considering the backtracking, we construct the binary tree, the values of nodes will be like [s,k,r].
Initially the root node looks like [0,1,73]



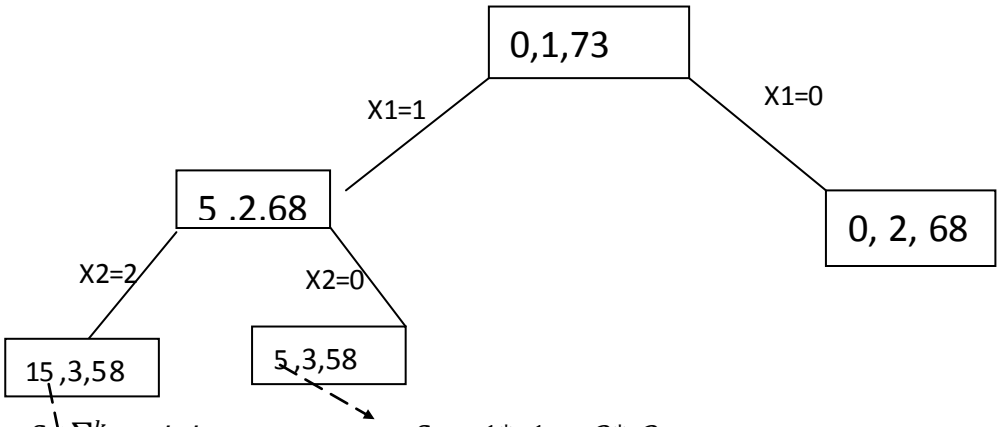
Now consider the x1 values ,now the tree will be expanding



S =$\sum_{i=1}^{k} wixi$
S=w1x1
=5*1
=5

s=w1x1
=5*0
=0

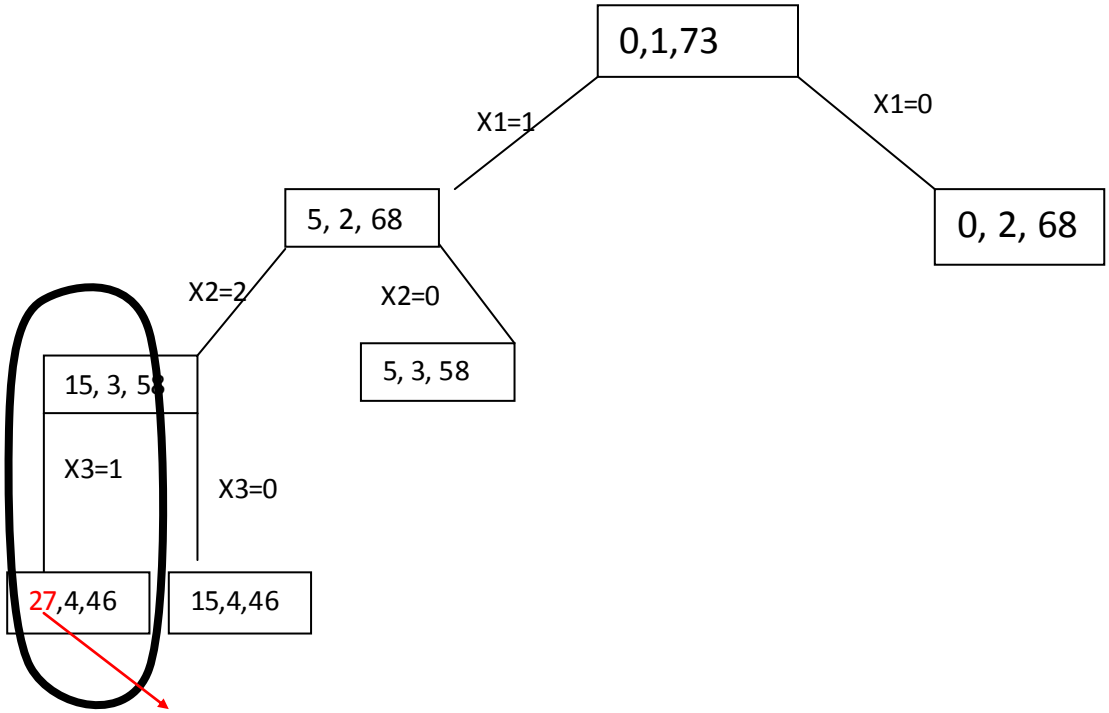
Now consider the x2 values ,and expand the left side of the tree,



S =$\sum_{i=1}^{k} wixi$
S=w1*x1 +w2*x2
=5*1 +10*1
=5+10
=15

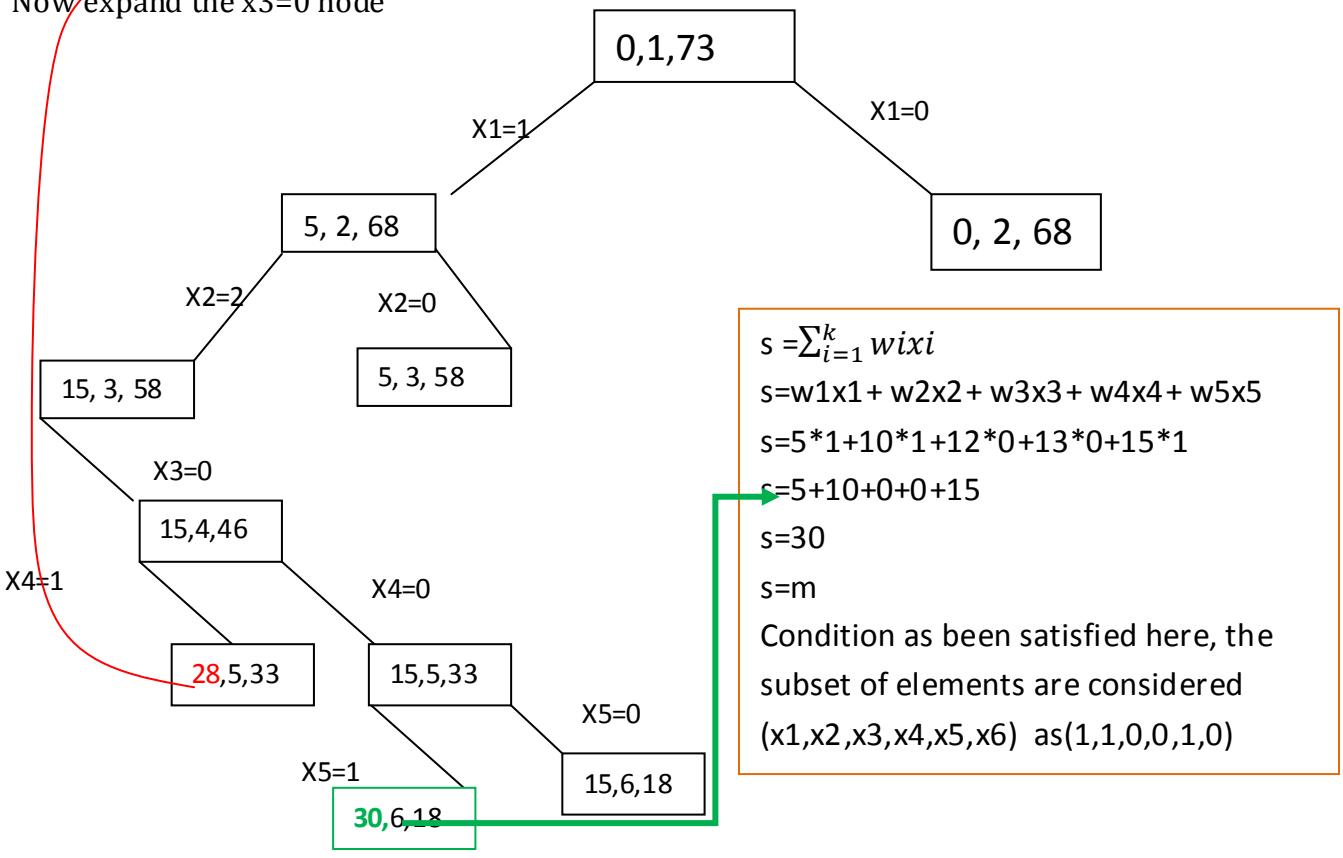
S= w1*x1 +w2*x2
=5*1+10*0
=5+0
=5

Similarly expand the tree by considering the values of x3=12, x4=13, x5=15, x6=18 on left side



The value of S is exceeding M, so we are not exploring the path further

Now expand the x3=0 node



$s = \sum_{i=1}^{k} wixi$

s=w1x1+ w2x2+ w3x3+ w4x4+ w5x5

s=5*1+10*1+12*0+13*0+15*1

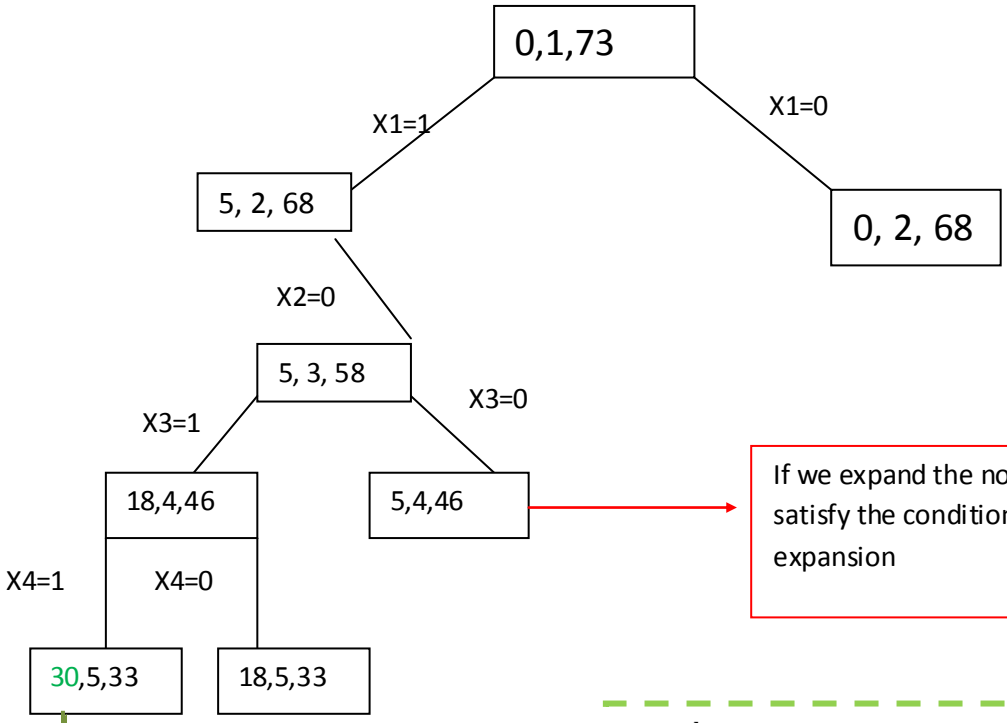
s=5+10+0+0+15

s=30

s=m

Condition as been satisfied here, the subset of elements are considered (x1,x2,x3,x4,x5,x6) as(1,1,0,0,1,0)

Now let us expand the node from x2=0 by using the backtracking



If we expand the node we could not satisfy the condition, so it is left without expansion

s =$\sum_{i=1}^{k} wixi$

s=w1x1+ w2x2+ w3x3+ w4x4

s=5*1+10*0+12*1+13*1

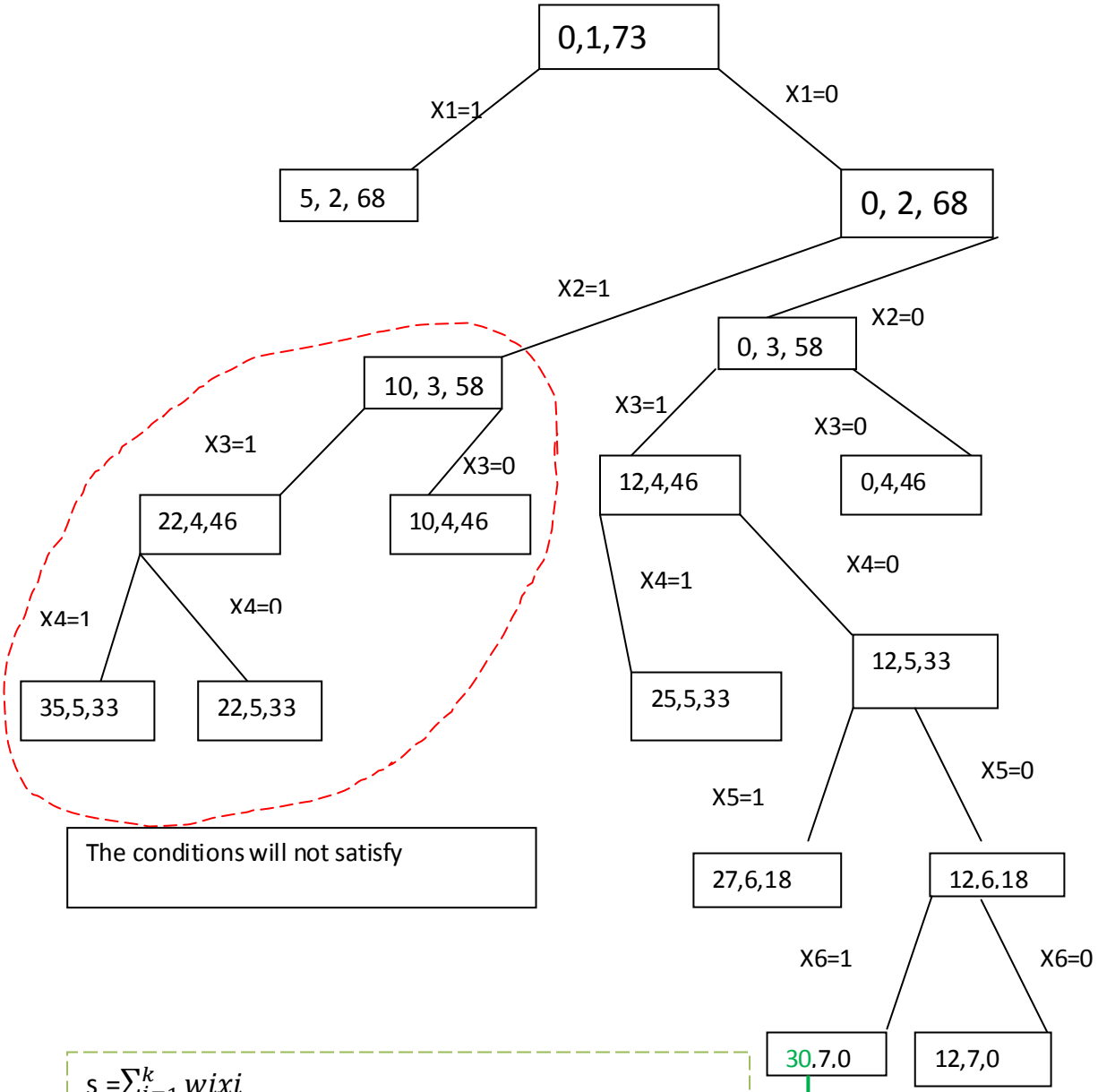
s=5+0+12+13

s=30

s=m

Condition as been satisfied here, the subset of elements are considered (x1,x2,x3,x4,x5,x6) as(1,0,1,1,0,0,0)

By using the backtracking we will start exploring from the node x1=0



s =$\sum_{i=1}^{k} wixi$

s=w1x1+ w2x2+ w3x3+ w4x4+w5x5+w6x6

s=5*0+10*0+12*1+13*0+15*0+18*1

s=0+0+12+0+0+18

s=30

s=m

Condition as been satisfied here, the subset of elements are considered (x1,x2,x3,x4,x5,x6) as(0,0,1,0,0,1)

So from the above given values there are 3 subset which results the m

*** **Subset Sum problem using** a **backtracking** approach which will take O(2^N) **time complexity**

GRAPH COLOURING

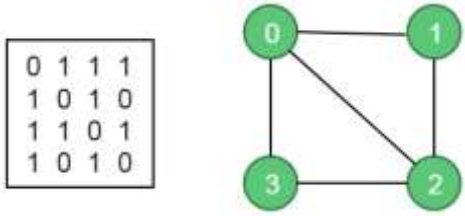
In this problem, an undirected graph is given. There is also provided m colors. The problem is to find if it is possible to assign nodes with m different colors, such that no two adjacent vertices of the graph are of the same colors. If the solution exists, then display which color is assigned on which vertex.

Starting from vertex 0, we will try to assign colors one by one to different nodes. But before assigning, we have to check whether the color is safe or not. A color is not safe whether adjacent vertices are containing the same color.

Input and Output

Input:

The adjacency matrix of a graph G(V, E) and an integer m, which indicates the maximum number of colors that can be used.



Let the maximum color m = 3.

Output:

This algorithm will return which node will be assigned with which color. If the solution is not possible, it will return false.

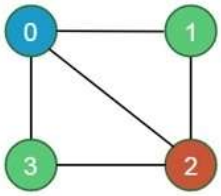
For this input the assigned colors are:

Node 0 -> color 1

Node 1 -> color 2

Node 2 -> color 3

Node 3 -> color 2



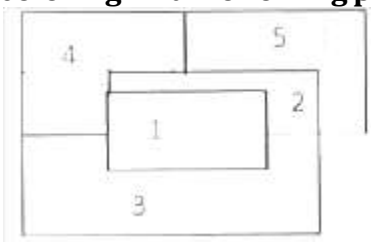
```
Algorithm m coloring(k)
{
        Repeat
        {
                Nextvalue(k);
                If(X[k]=0) then
                        Return; // no new color is possible
                If(k== N) then
                        Write(x[1:n]) // At most m color have been used to color n vartices
                Else
                        Mcoloring(k+1);
        }
        Until(false);
}
Algorithm Nextvalue(k)
{
        Repeat
        {
        X[k]:=(x[k]+1)mod(m+1) //next highest color
        If (x[k]=0) then
                Return; // all color haven been used
                For j=1 to n do
                {
                        // check if this color is different from adjacent color or not
                        If((g[k,j] !=0) and (x[k] =x[j]) then
                        // if (k,j] is an edge & if adjacent vertices have the same  color
                                Break;
                }
                If(j=n+1) then
                        Return; // new color found
        }until(false);
        // otherwise try to find another color.
}
```

Questions

1. What is weighting rule for Union(i,j)? How it improves the performance of union operation?
2. Explain the Find algorithm with collapsing rule
3. Describe Union and Find algorithms
4. What is a set? List the operations that can be performed on it.
5. **List the disjoint set operations and explain with examples**.

6. Explain the 4-queen problem using backtracking.
7. **Briefly explain n-queen problem using backtracking.**
8. Explain the 4-queen problem using backtracking.
9. Give the solution to *-queen problem using backtracking

10. Draw the state space tree for m-coloring graph.
11. Describe Backtracking technique to m-coloring graph.
12. **What is graph coloring problem? Describe the back tracking technique to m-coloring With following planar graph shown below**



13. Draw the state space tree for m-coloring graph
14. Give brief note on graph coloring.

15. **What is sum-of-subsets problem? Write a recursive backtracking algorithm for sum of subsets problem**.
16. **Define Backtracking? List the applications of Backtracking**